

Smartphone-Based Fall Detection and Fall-Type Classification

Mrunal Kotkar¹, Sushant Lokhande²

¹ mrunalsuhas.kotkar@sjsu.edu, Computer Science, San Jose State University, San Jose, CA, USA.

² sushantrakesh.lokhande@sjsu.edu, Computer Science, San Jose State University, San Jose, CA, USA.

Keywords: fall detection, human activity recognition, smartphone sensors, classical machine learning, deep learning, transfer learning.

Abstract

This project investigates smartphone-based fall detection and activity recognition using the MobiFall Dataset v2.0. The study considers three tasks: binary fall detection, four-class fall-type classification, and 13-class activity recognition. Raw sensor streams are synchronized, resampled to 100 Hz, segmented into fixed windows, and represented either as handcrafted feature vectors, raw multichannel sequences, or spectrogram images. Classical machine learning models are used as baselines, while ANN, CNN, and LSTM models are trained directly on the raw sensor data. Transfer learning is further explored through a pretrained DeepCNN model on UCI HAR and image-based models such as ResNet50 and VGG16 on spectrogram representations. The results show that the 1D CNN on raw 3-second windows achieves the best overall performance for binary fall detection, while LOSO and LOEO evaluation confirm strong generalization.

1 Introduction & Methodology

1.1 Motivation and Project Objectives

Falls are a major health risk, especially for elderly populations, where delayed detection can lead to serious injury or complications. With the widespread availability of smartphones equipped with motion sensors, fall detection systems have become a practical and scalable direction for wearable artificial intelligence. The main challenge is to distinguish true fall events from normal daily activities such as walking, sitting, jogging, or stair movement, which may produce similar motion patterns.

In our previous work (Demo 1), fall detection was performed using pre-computed features and classical machine learning models. While that approach achieved strong baseline performance, it depended heavily on manually engineered features and could not fully exploit the temporal structure present in raw sensor signals.

In this phase (Demo 2), we transition to the MobiFall Dataset v2.0, which provides raw accelerometer and gyroscope time-series recordings. This allows us to study both classical machine learning baselines and deep learning models within a common experimental framework. The project focuses on three tasks:

- Binary classification: fall vs non-fall
- Fall-type classification: 4-class problem
- Activity recognition: 13-class problem

In addition, we explore transfer learning to evaluate whether motion representations learned from related datasets or alternative input domains can improve fall detection performance or training efficiency.

1.2 Dataset Overview and Pipeline

The MobiFall Dataset v2.0 contains smartphone-based accelerometer and gyroscope signals for fall events and daily activities. Due to variable durations and irregular sampling, a unified preprocessing and modeling pipeline is required.

Table 1: MobiFall v2.0 dataset structure summary

Attribute	Value
Dataset organization	Subject-wise folders (sub1, sub2, ...)
Filename format	<activity>_<sensor>_<subject>_<trial>.txt
Total subjects	24
ADL classes	9
Fall classes	4
Sensors	Accelerometer, gyroscope, orientation
Accelerometer fields	timestamp(ns), x, y, z (m/s ²)
Gyroscope fields	timestamp(ns), x, y, z (rad/s)
Orientation fields	timestamp(ns), azimuth, pitch, roll
Sampling frequency	Accelerometer: 87 Hz; Gyroscope/Orientation: 100 Hz

Table 2: Activity labels and their fall/non-fall mapping

Code	Full Name	Category
SDL	Sideward-lying	Fall
FOL	Forward-lying	Fall
FKL	Forward Knee-lying	Fall
BSC	Back Sitting Chair	Fall
STD	Standing	Not-Fall
WAL	Walking	Not-Fall
JOG	Jogging	Not-Fall
JUM	Jumping	Not-Fall
STU	Stairs Up	Not-Fall
STN	Stairs Down	Not-Fall
SCH	Sit Chair	Not-Fall
CSI	Car-step In	Not-Fall
CSO	Car-step Out	Not-Fall

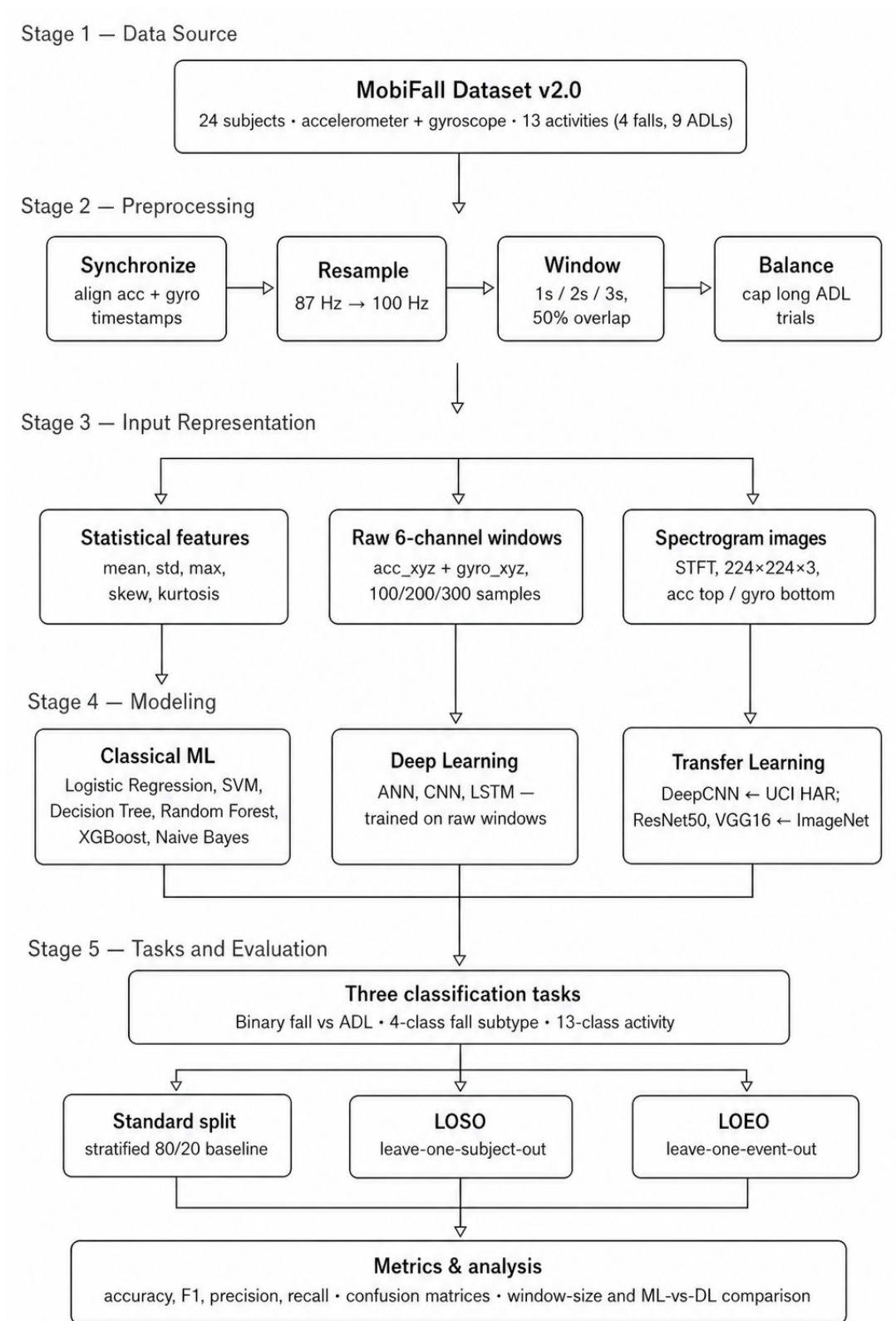


Figure 1: Project Pipeline

Figure 1 outlines the workflow, consisting of five stages: data source, preprocessing, input representation, modeling, and evaluation. Stages 3 and 4 branch into three model families and merge at evaluation.

1.3 Methodology Overview

Stage 1: Data Source. The dataset includes 24 subjects performing 9 ADLs and 4 fall types, with six-channel inertial data sampled at ~ 200 Hz (with jitter). Only 9 subjects contain both falls and ADLs, constraining ADL-related tasks, while all 24 are used for fall-type classification.

Stage 2: Preprocessing. Signals are synchronized, resampled to 100 Hz, and segmented into 1s, 2s, and 3s windows with 50% overlap. Long ADL recordings are capped to reduce class imbalance.

Stage 3: Input Representation. Three representations are used: (i) handcrafted statistical features for classical ML, (ii) raw multichannel time series for deep learning, and (iii) spectrogram images via STFT for transfer learning.

Stage 4: Modeling. Three model families are explored: classical ML (LR, SVM, RF, XGBoost), deep learning (ANN, CNN, LSTM), and transfer learning using pretrained CNNs (UCI HAR and ImageNet).

Stage 5: Tasks and Evaluation. Models are evaluated on binary fall detection (2 classes), fall-type classification (4 classes), and 13-class activity recognition. Evaluation uses stratified split, LOSO, and LOEO protocols. Performance is measured using accuracy, precision, recall, and F1-score (primary), with confusion matrices for detailed analysis.

2 Literature Review

Recent literature on fall detection shows a clear shift from threshold-based and handcrafted-feature methods toward deep learning models that operate directly on inertial time-series data collected from smartphones and wearable devices. The selected studies cover the main dimensions relevant to this project: benchmark datasets such as MobiFall, feature engineering, deep sequence models such as CNN/LSTM/Transformer, transfer learning, cross-dataset generalization, and real-world deployment challenges such as false positives and sampling-rate sensitivity. Together, these papers provide both the methodological background and the practical motivation for evaluating machine learning and deep learning models on the MobiFall v2 dataset.

Table 3: Literature Review 1

Ref.	Paper	Key Idea	Model(s)	Dataset(s)	Metrics / Main Results	Limitations / Relevance
[1]	Vavoulas et al. (2014), <i>The MobiFall Dataset: Fall Detection and Classification with a Smartphone</i>	Introduces the MobiFall dataset for smartphone-based fall detection and classification using inertial signals.	Dataset / benchmark paper	MobiFall	Provides benchmark dataset for falls and ADLs rather than a new SOTA model.	Not recent, but it is the canonical citation for the dataset used in this project. Essential for describing data source and activity/fall categories.
[2]	Yhdego et al. (2023), <i>Toward Real-Time, Robust Wearable Sensor Fall Detection Using Deep Learning Methods: A Feasibility Study</i>	Studies sliding-window segmentation, class imbalance handling, and sensor-position analysis for real-time fall detection.	CNN + LSTM + Transformer-style hybrid deep model	Wearable multi-position sensor datasets	Reported best F1-score of 0.97, with shank placement identified as the best sensor position.	Uses wearable body-mounted sensors rather than smartphone-only data, but highly relevant for windowing strategy, imbalance treatment, and deep architecture design.
[3]	Liu et al. (2023), <i>A Review of Wearable Sensors Based Fall-Related Recognition Systems</i>	Comprehensive review of wearable fall prediction, detection, and classification systems.	Survey / review	Multiple public datasets	No single benchmark metric; synthesizes methods, sensors, and algorithm trends.	Useful as a literature-positioning paper, but not a direct experimental baseline. Good for introduction and identifying research gaps.
[4]	Kim et al. (2023), <i>Detection of Important Features and Comparison of Datasets for Fall Detection Based on Wrist-Wearable Devices</i>	Selects important statistical features and compares fall-detection performance across datasets.	Feature selection + classical ML	Wrist-wearable fall datasets	Selected 15 important features and achieved high accuracy with reduced feature sets.	Focused on wrist-worn devices, not smartphone-in-pocket settings. Still very useful for motivating feature engineering and dataset sensitivity.
[5]	Fula and Moreno (2024), <i>Wrist-Based Fall Detection: Towards Generalization across Datasets</i>	Addresses cross-dataset generalization and robustness by combining wrist-based datasets.	ANN and SVM-based models with imbalance-aware training	Combined wrist-based datasets	Reported recall of 90.57%, specificity of 96.91%, and AUC of 98.85% on combined datasets.	Device placement differs from smartphone waist/pocket sensing, but the paper is highly relevant for discussing generalization, class imbalance, and evaluation beyond single-dataset results.

Table 4: Literature Review 2

Ref.	Paper	Key Idea	Model(s)	Dataset(s)	Metrics / Main Results	Limitations / Relevance
[6]	Li et al. (2024), <i>An Effective Deep Learning Framework for Fall Detection: Model Development and Validation</i>	Proposes a deep learning framework that emphasizes informative temporal segments for fall detection.	DSCS deep learning model	SisFall and MobiFall	On MobiFall, reported 99.65% accuracy, 100% recall, and 98.39% precision.	Excellent directly related paper because it evaluates on MobiFall. Limitation: very high controlled-setting performance may not fully transfer to noisy real-world deployment.
[7]	Haque et al. (2024), <i>Experimental Study of Long Short-Term Memory and Transformer Models for Fall Detection on Smartwatches</i>	Compares LSTM and Transformer models for smartwatch fall detection, with emphasis on real-time behavior.	LSTM variants and Transformer	Smartwatch fall datasets	Uses precision, recall, and F1-score; reports that Transformer gives better recall and fewer false positives in real-time testing.	Smartwatch-based rather than smartphone-based, but very relevant to the ANN/CNN/LSTM comparison and future advanced-model discussion.
[8]	Paolini et al. (2024), <i>Deep Transfer Learning Approach in Smartwatch-Based Fall Detection Systems</i>	Transforms time-series accelerometer windows into scalograms and applies transfer learning with pre-trained deep models.	Transfer learning on deep image models using CWT scalograms	WEDA-FALL	Reported approximately 98% accuracy in preliminary tests.	Dataset is not MobiFall, and results are preliminary, but it is highly relevant for the transfer-learning section because it shows a practical transfer-learning pipeline for fall detection.
[9]	Ngu et al. (2025), <i>Enhancing Real-World Fall Detection Using Commodity Devices: A Systematic Study</i>	Examines the gap between offline accuracy and real-world performance on phones and watches; retraining with feedback improves deployment quality.	Transformer-based model with multi-sensor combinations	Commodity smartphone + smartwatch data	Reported best offline F1-score of 0.96; real-world feedback-based refinement achieved F1-score of 0.92 with fewer false positives.	Very relevant because it highlights that strong offline results may still fail in real use. Supports discussion of false alarms, deployment realism, and evaluation rigor.
[10]	Villa and Casilari (2026), <i>The Impact of the Accelerometer Sampling Rate on the Performance of Machine and Deep Learning Models in Wearable Fall-Detection Systems</i>	Systematically studies how sampling frequency affects accuracy, stability, and computational efficiency.	Multiple ML and DL models	Wearable accelerometer fall datasets	Focuses on the trade-off between performance and computational cost across sampling rates.	Directly useful for justifying resampling and pre-processing decisions in the pipeline, even though it is not specific to MobiFall.

Across these studies, three themes are especially relevant to our work. First, model performance depends strongly on preprocessing choices such as resampling, segmentation, and class balancing. Second, deep learning models generally outperform handcrafted pipelines when raw time-series data is available. Third, strong offline accuracy does not always imply strong generalization in realistic settings, which motivates our use of LOSO and LOEO evaluation.

3 Classical Machine Learning Implementation

3.1 Feature-Based Dataset Preparation

To establish baseline performance, classical machine learning models were trained on hand-crafted window-level features derived from the preprocessed MobiFall signals. The same core preprocessing pipeline described in Section 1 was used so that comparisons with deep learning models remain fair. Specifically, the accelerometer and gyroscope signals were synchronized, resampled to 100 Hz, segmented into fixed windows, and class-balanced by limiting overly long non-fall recordings.

Since traditional machine learning methods cannot directly process raw variable-length sequences, each window was converted into a fixed-length feature vector. Statistical and domain-relevant features were extracted from the six sensor channels, including mean, standard deviation, maximum value, skewness, and kurtosis. This representation provides a compact summary of motion behavior while remaining compatible with tabular classifiers.

3.2 Models Used

The following classical models were evaluated:

- Logistic Regression
- Naive Bayes
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- XGBoost

These models were selected to cover linear, non-linear, tree-based, and ensemble-based approaches. They serve as strong baselines for later comparison with deep learning models.

3.3 Results and Discussion

Table 5: ML model results for Binary Fall Detection

Model	Accuracy	F1-Score	Precision	Recall
XGBoost	0.9509	0.9562	0.9361	0.9771
SVM	0.9452	0.9513	0.9269	0.9771
Random Forest	0.9292	0.9381	0.9004	0.9792
Logistic Regression	0.9007	0.9133	0.8757	0.9542
Decision Tree	0.8607	0.8806	0.8303	0.9375
Naive Bayes	0.8196	0.8364	0.8313	0.8417

Table 5 shows that XGBoost achieved the strongest classical baseline, with an accuracy of 0.9509 and an F1-score of 0.9562. SVM also performed competitively, while ensemble methods consistently outperformed simpler models such as Decision Tree and Naive Bayes. These results suggest that feature-based fall detection is effective, but also indicate that more expressive models are needed to capture the complex temporal structure of raw motion signals.

4 Deep Learning Implementation

4.1 Data Preprocessing and Input Representation

In this phase, we worked directly with MobiFall v2.0 raw inertial signals. Each recording is stored as a text file containing timestamps and sensor readings, with variable durations and irregular sampling characteristics. To prepare the data for neural networks, we applied the following steps:

- **Sensor fusion:** accelerometer (a_x, a_y, a_z) and gyroscope (g_x, g_y, g_z) signals were combined to form a 6-channel input.
- **Resampling:** all signals were resampled to a uniform 100 Hz frequency.
- **Windowing:** each recording was segmented into fixed-length windows of 1 s, 2 s, and 3 s.
- **Imbalance handling:** overly long non-fall activities were capped to reduce majority-class bias.

After preprocessing, the 3-second configuration produced approximately 3800 training samples. Two input representations were studied:

- **Raw time-series windows** for ANN, CNN, and LSTM
- **Spectrogram images** for image-based transfer learning

4.2 Model Development

Three deep learning architectures were implemented and compared: ANN, CNN, and LSTM. All models used the same six-channel input representation, allowing a fair comparison between fully connected, convolutional, and recurrent approaches.

- **Artificial Neural Network (ANN):** the ANN uses flattened windows followed by fully connected layers. It serves as a simple baseline, but it does not explicitly model temporal order.
- **Convolutional Neural Network (CNN):** the CNN applies 1D convolution and pooling to the raw time-series windows, allowing it to capture local motion patterns and short-term temporal structure.
- **Long Short-Term Memory (LSTM):** the LSTM processes the input as a sequence and is intended to model temporal dependencies across time steps.

Table 6: Deep learning architectures

Model	Architecture Details	Total Params	Key Characteristics
ANN	Flatten $\rightarrow$ Dense(256) $\rightarrow$ Dropout $\rightarrow$ Dense(128) $\rightarrow$ Dropout $\rightarrow$ Output(1)	340,481	Simple baseline; ignores temporal order but trains quickly
CNN	Conv1D(64) $\rightarrow$ BN $\rightarrow$ MaxPool $\rightarrow$ Conv1D(128) $\rightarrow$ BN $\rightarrow$ MaxPool $\rightarrow$ Conv1D(128) $\rightarrow$ GAP $\rightarrow$ Dropout $\rightarrow$ Dense(64) $\rightarrow$ Output(1)	101,441	Captures local temporal patterns; best trade-off between accuracy and efficiency
LSTM	LSTM(64, return seq) $\rightarrow$ Dropout $\rightarrow$ LSTM(64) $\rightarrow$ Dropout $\rightarrow$ Dense(32) $\rightarrow$ Output(1)	53,313	Models sequence structure; slower training and limited gain for short windows

4.3 Training Setup and Behavior

To ensure a fair comparison, all models were trained under a common setup. The optimizer was Adam or AdamW with an initial learning rate of 10^{-3} . Binary classification used binary cross-entropy, while fall-type and multiclass activity recognition used sparse categorical cross-entropy. Models were trained with batch size 64 for up to 50 epochs, with early stopping and learning-rate reduction on validation plateau.

Table 7: Training configuration for deep learning models.

Parameter	ANN	CNN	LSTM
Optimizer	Adam / AdamW	Adam / AdamW	Adam / AdamW
Learning Rate	1×10^{-3}	1×10^{-3}	1×10^{-3}
Batch Size	64	64	64
Epochs	20–50	20–50	20–50
Regularization	Dropout	Dropout	Dropout
Data Augmentation	Noise + Time Shift	Noise + Time Shift	Noise + Time Shift

During training, the ANN converged quickly and served as a lightweight baseline, but it could not exploit sequential structure effectively. The CNN showed the most stable optimization behavior and generalized best across window sizes. The LSTM captured temporal dynamics but required significantly more training time and did not outperform the CNN on this dataset.

4.4 Results on Standard Split

Table 8: Deep Learning Model Results for Binary Fall Detection

Window	Model	Accuracy	F1-Score	Epochs	Training Time (s)
1s	ANN	0.9167	0.9166	50	51.8
1s	CNN	0.9281	0.9280	20	151.0
1s	LSTM	0.9295	0.9294	46	805.0
2s	ANN	0.9396	0.9375	25	16.8
2s	CNN	0.9753	0.9742	44	304.8
2s	LSTM	0.9337	0.9304	31	576.1
3s	ANN	0.9699	0.9677	13	8.1
3s	CNN	0.9935	0.9931	32	229.1
3s	LSTM	0.9516	0.9494	20	371.9

Table 9: Fall-type classification (4-class) performance for 3-second windows.

Model	Accuracy	F1-Score	Epochs	Time (s)
ANN	0.6332	0.6388	20	8.2
CNN	0.8062	0.8076	50	129.9
LSTM	0.6609	0.6619	36	237.1

Table 10: Multiclass activity recognition (13-class) performance for 3-second windows.

Model	Accuracy	F1-Score	Epochs	Time (s)
ANN	0.7843	0.7810	42	27.2
CNN	0.9111	0.9236	50	355.9
LSTM	0.7647	0.6917	50	892.3

The deep learning results show three consistent trends. First, performance improves as the window size increases, indicating that longer temporal context helps capture the full motion pattern of a fall. Second, the CNN consistently outperforms ANN and LSTM across all three tasks, with the gap widening as task difficulty grows — the CNN exceeds the next-best model by roughly 1% on binary classification, but by 14–17% on the fall-type and 13-class tasks. Third, the LSTM incurs the largest training cost without delivering better performance than the CNN, suggesting that explicit recurrence offers limited benefit over convolution at this window length. Based on these results, the CNN with 3-second raw windows is selected as the strongest direct deep learning model in this study.

Confusion matrices — Binary (Fall vs ADL)

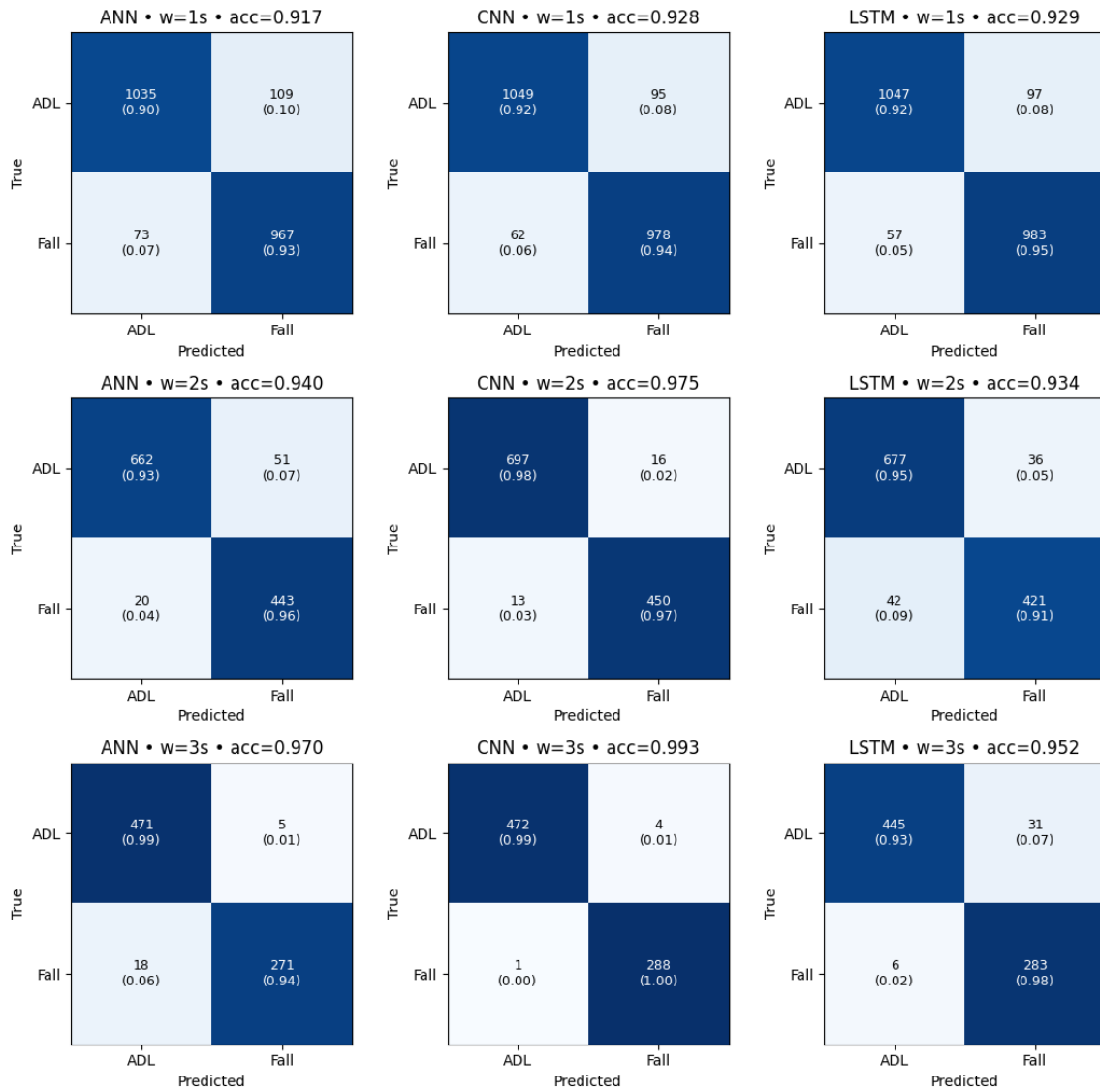


Figure 2: Confusion matrices for binary fall detection

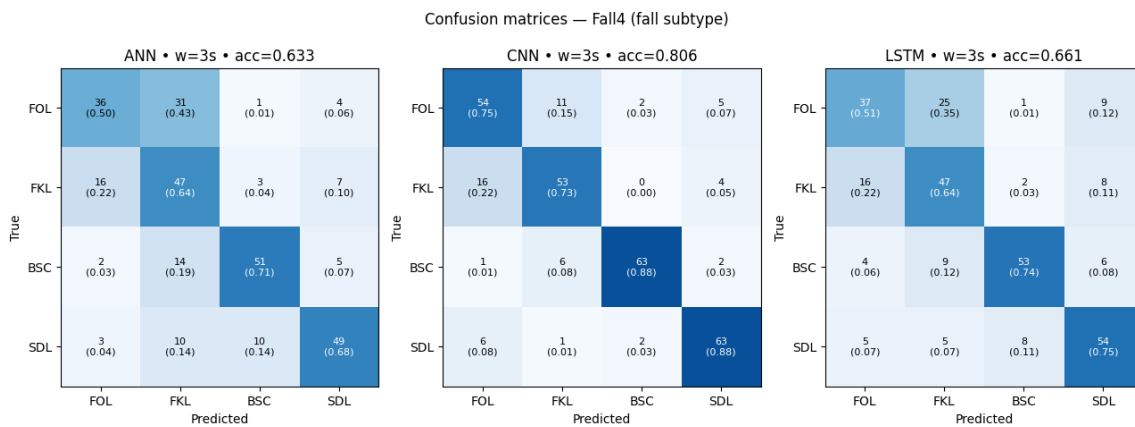


Figure 3: Confusion matrices for fall-type classification (4-class)

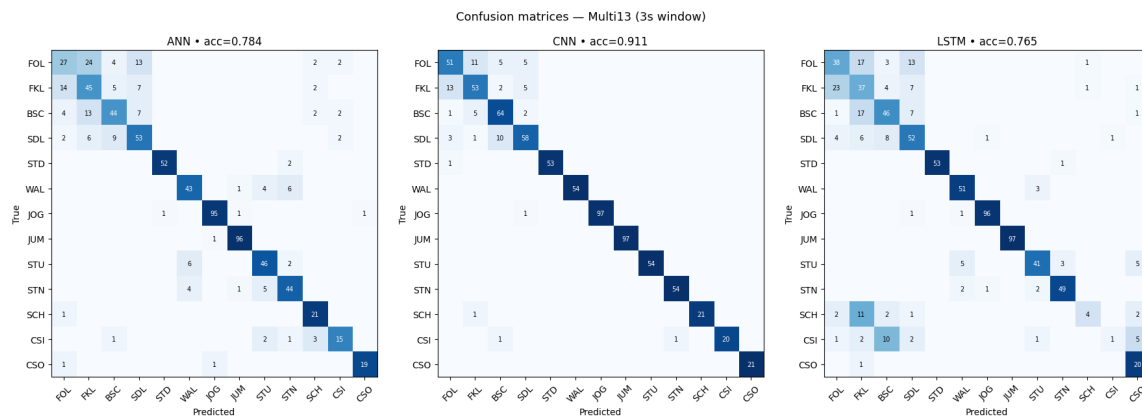


Figure 4: Confusion matrices for activity recognition (13-class)

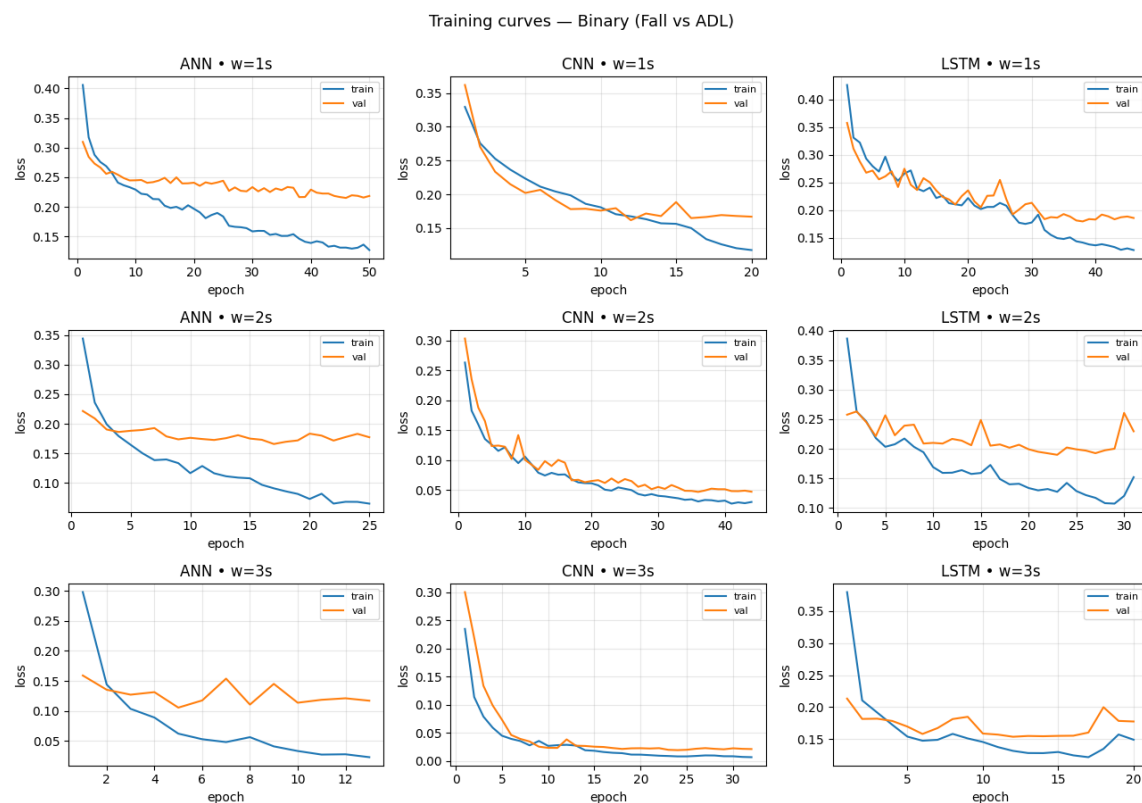


Figure 5: Training and validation loss curves for binary fall detection

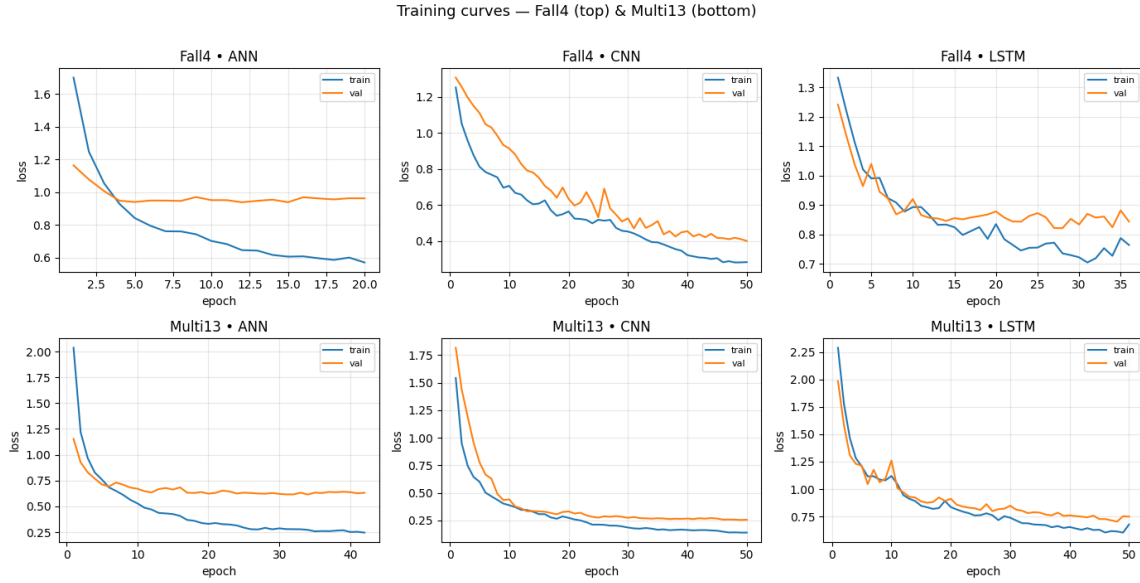


Figure 6: Training and validation loss curves for the fall-type and 13-class detection

5 Advanced Modeling & Transfer Learning

Deep learning models often benefit from larger datasets than MobiFall alone can provide. To study whether learned motion representations can be reused across domains, we explored transfer learning in two forms: time-series transfer from UCI HAR and image-based transfer from spectrogram representations.

5.1 UCI HAR Pretraining

Direct training on MobiFall is constrained by the relatively small number of subjects (24) and the even smaller subset (9) usable for fall-vs-ADL classification. To compensate, we adopt a transfer learning strategy: pretrain a deeper convolutional network on a larger, related human-activity dataset, then transfer its learned features to MobiFall. We chose UCI HAR as the source dataset because it shares the same sensor modality (smartphone accelerometer + gyroscope), uses comparable body placement, and provides 7,352 training windows from 21 subjects performing six daily activities.

Source-domain architecture. For the pretraining backbone we designed a deeper 1D convolutional network, **DeepCNN_HAR**, with approximately 1 million parameters, roughly $10\times$ the capacity of the simple CNN used in Section 4. The added depth is motivated by the observation that low-level convolutional features (envelope detection, periodicity, edge transitions) are the most likely to transfer across human-motion datasets, while task-specific classifiers must be re-learned. Building a strong source model first ensures that the transferred features carry useful signal. The architecture is summarized in Table 11; key choices include residual (skip) connections to enable training depth without vanishing gradients, separable convolutions in the deeper blocks for parameter efficiency, and global average pooling rather than flattening to produce a fixed-size 256-dimensional feature vector regardless of input length.

Table 11: Summary of the DeepCNN_HAR architecture used for UCI HAR pretraining.

Component	Description
Input	Time-series window of shape (128×6)
Early feature extraction	Conv1D(64) + BN + ReLU + SpatialDropout
Intermediate blocks	Conv1D(128) + BN + ReLU + SpatialDropout + MaxPooling
Residual learning	Residual/skip connections across convolutional stages
Efficient convolutions	SeparableConv1D layers in residual blocks
Deep feature stage	Conv1D / SeparableConv1D blocks with 256 filters
Temporal compression	GlobalAveragePooling1D to obtain a 256-dimensional feature vector
Classifier head	Dropout + Dense(128) + Dropout + Dense(6, softmax)
Total parameters	1,009,862

Pretraining setup. The network was trained on UCI HAR using AdamW (initial learning rate 10^{-3} , weight decay 10^{-4}), batch size 64, and lightweight augmentation (Gaussian noise plus random time shifts of up to 8 samples). Training ran for up to 60 epochs with early stopping on validation accuracy. Per-channel z-score normalization was fitted on the training split only to avoid leakage into the test set.

Table 12: Performance of DeepCNN_HAR on the UCI HAR test set.

Activity	Precision	Recall	F1-Score
WALKING	0.9979	0.9456	0.9710
WALK_UP	1.0000	0.9448	0.9716
WALK_DN	0.8955	1.0000	0.9449
SITTING	0.9381	0.8024	0.8650
STANDING	0.8453	0.9549	0.8967
LAYING	0.9908	1.0000	0.9954
Macro Avg	0.9446	0.9413	0.9408
Weighted Avg	0.9448	0.9410	0.9407

The pretrained model reached **94.10% test accuracy** on the held-out UCI HAR subjects (Table 12), which is comparable to published results on this dataset. Per-class analysis is informative: locomotion classes (WALKING, WALK_UP, WALK_DN, LAYING) achieve F1 above 0.94, while the static postures SITTING and STANDING are confused with each other (F1 = 0.86 and 0.90 respectively) because they differ only in phone orientation rather than motion. With the network demonstrating strong general-purpose motion features, we then transfer these features to MobiFall.

5.2 Fine-Tuning on MobiFall

Transferring DeepCNN_HAR to MobiFall involves three steps:

1. **Architectural transplant.** Because the convolutional layers use `padding="same"` and the feature aggregation is global average pooling, the backbone is shape-agnostic. It operates equally well on the 128-sample UCI HAR windows it was trained on and the 100/200/300-sample MobiFall windows it now sees. We rebuild the same backbone with MobiFall’s input shape and copy the learned weights layer by layer, preserving all 25 convolutional and batch-normalization layers. Only the classification head is replaced (with a fresh head sized to the new task: 1 unit for binary, 4 for fall-type, 13 for multi-class).
2. **Stage 1: frozen backbone.** The transferred backbone is frozen and only the new head is trained at learning rate 10^{-3} for up to 15 epochs. This step lets the head learn to map UCI HAR features onto MobiFall labels without disturbing the pretrained features themselves.
3. **Stage 2: end-to-end fine-tuning.** The backbone is unfrozen and the entire network is trained at a $10\times$ smaller learning rate (10^{-4}) for up to 40 epochs. The reduced learning rate prevents catastrophic forgetting of the pretrained features while still allowing the upper layers to specialize to MobiFall.

5.3 Resultts of Training on UCI HAR Dataset and Fine-Tuning on MobiFall

On binary fall detection, transfer learning reaches 99.09% accuracy, statistically indistinguishable from the 99.35% achieved by training the CNN from scratch, while requiring less than half the training time. However, on the harder fall-type and 13-class tasks (Table 14), transfer from UCI HAR consistently *underperforms* the from-scratch CNN: 60.6% vs 80.6% on fall4, and 84.2% vs 91.1% on multi13. This is a documented failure mode known as *negative transfer*: because UCI HAR contains no fall events, its convolutional features are tuned to discriminate between sustained, periodic activities (walking, sitting, laying) and effectively smooth over the brief impact peaks that characterize falls. The pretrained representation therefore helps on the easy task where the impact signal is large and unmistakable, but actively hurts on tasks that require fine-grained discrimination among similar fall events.

Table 13: Transfer learning vs. from-scratch CNN on binary fall detection (3-second windows).

Model	Accuracy	F1-Score	Training Time (s)
CNN (No Transfer)	0.9935	0.9931	229.1
CNN (Transfer Learning)	0.9909	0.9903	110.0

Table 14: Transfer learning performance (UCI HAR $\rightarrow$ MobiFall) across tasks and window sizes.

Window (s)	Task	Accuracy	F1-Score	Epochs (Stage 1 + Stage 2)	Time (s)
1	Binary	0.9304	0.9303	15+40	131.9
1	Fall4	0.6183	0.6049	15+40	100.0
1	Multi13	0.6690	0.6846	15+40	127.7
2	Binary	0.9651	0.9634	15+40	124.6
2	Fall4	0.6263	0.6169	9+40	96.9
2	Multi13	0.7925	0.7830	15+40	117.0
3	Binary	0.9909	0.9903	15+40	110.0
3	Fall4	0.6055	0.6107	8+40	99.6
3	Multi13	0.8418	0.8434	15+40	112.7

5.4 Spectrogram-Based Transfer Learning

A second transfer-learning strategy bridges the gap between time-series sensor data and the image-based ImageNet pretraining used by ResNet50 and VGG16. Both networks expect a $224 \times 224 \times 3$ image as input, so the raw multichannel signal must first be transformed into an image-like representation that preserves the structure of motion events. **Signal-to-image conversion.** Each 3-second window is converted into an image through four steps:

1. **Compute the spectrogram.** A Short-Time Fourier Transform (STFT) is applied independently to each of the six sensor channels (accelerometer x, y, z and gyroscope x, y, z). The STFT slides a small analysis window of length 64 samples across the signal with 48 samples of overlap, and reports the strength of each frequency component within that window. The output is a 2-D map: time on one axis, frequency on the other.
2. **Convert to decibels.** Raw spectral magnitudes span several orders of magnitude, which would map to nearly black images. We compress the dynamic range using:

$$X_{\text{dB}} = 20 \log_{10}(|X| + \epsilon)$$

where X is the spectrogram magnitude and $\epsilon = 10^{-8}$ prevents $\log(0)$. Values below -40 dB are clipped, since they correspond to noise-floor energy.

3. **Normalize per channel.** Each spectrogram is rescaled to the $[0, 255]$ range using its 2nd–98th percentile values rather than its absolute min and max. This makes the contrast robust to outliers while preserving meaningful structure.
4. **Stack into a 3-channel image.** The three accelerometer spectrograms become the R, G, B channels of one image, and the three gyroscope spectrograms form a second image. The two are stacked vertically (accelerometer on top, gyroscope on bottom) and resized to 224×224 pixels — the input shape expected by ResNet50 and VGG16.

Figure 7 shows representative composite spectrograms for fall and ADL windows, and Figure 8 pairs the raw signal of selected windows with the corresponding spectrogram. Falls show a quiet baseline followed by a sharp impact spike, which appears in the spectrogram as a bright vertical broadband stripe; periodic ADLs such as walking and jogging produce repeating vertical patterns instead.

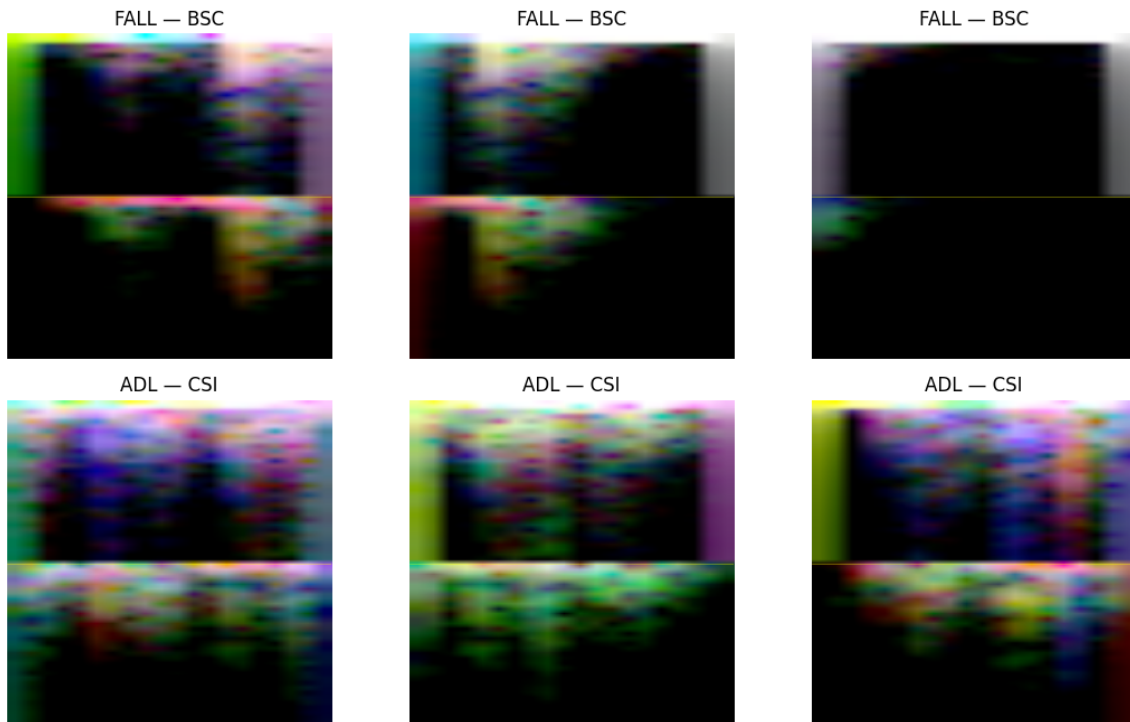


Figure 7: Composite spectrogram images used as input to ResNet50 and VGG16. Top row: backward-into-chair (BSC) falls. Bottom row: car-step-in (CSI) ADL trials. Each image’s top half is the accelerometer spectrogram ($RGB = x, y, z$) and the bottom half is the gyroscope spectrogram. Falls show concentrated impact energy followed by silence; ADLs show distributed periodic structure.

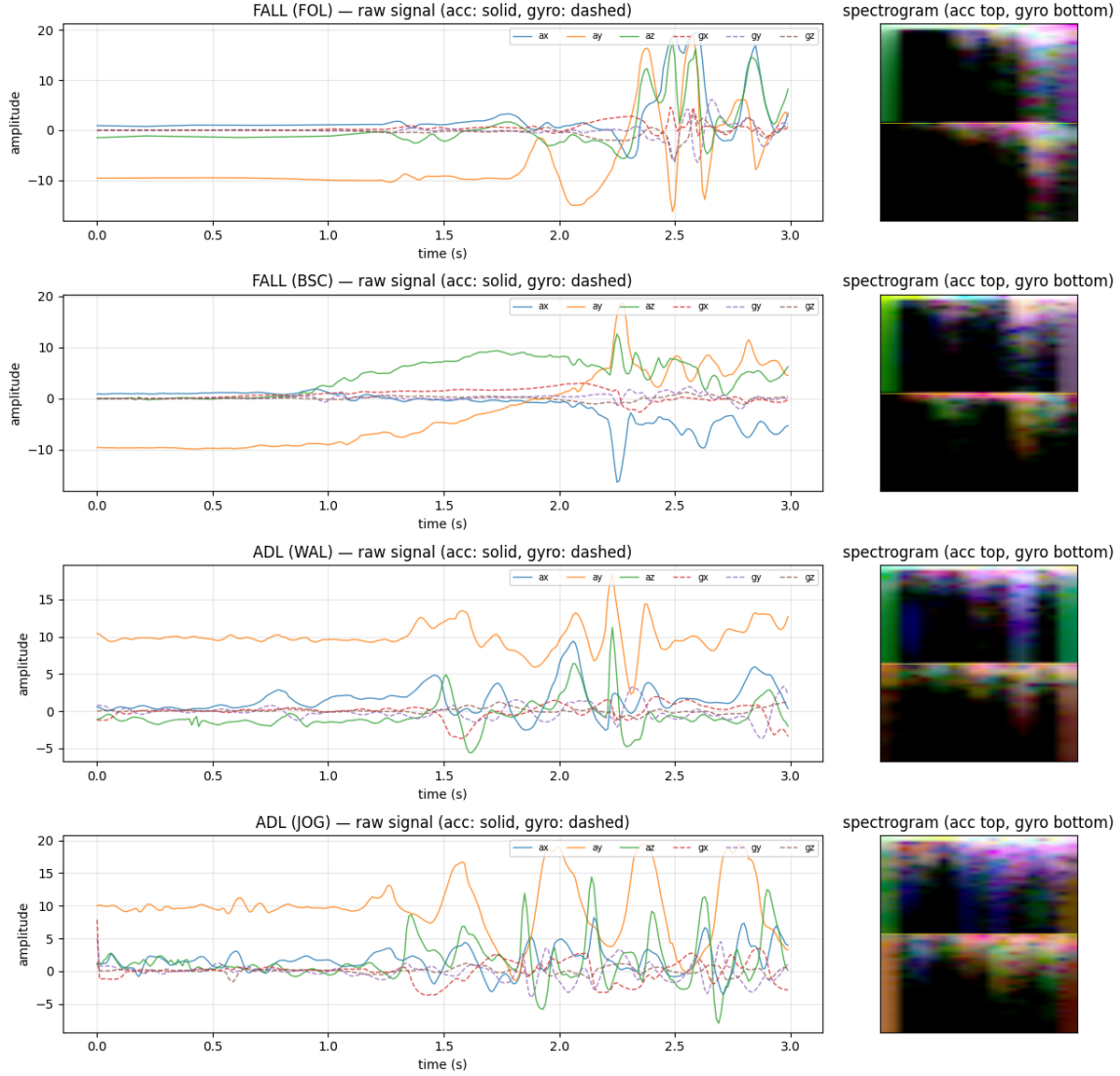


Figure 8: Raw 3-second sensor windows (left) and the corresponding spectrogram images (right) fed to the ImageNet backbones. Sharp transient events in the raw signal map to localized broadband energy in the spectrogram, while periodic activities produce repeating vertical structure.

Why this works. ResNet50 and VGG16 were pretrained on ImageNet to recognize edges, textures, and periodic patterns in natural photographs. These same low-level features are useful for spectrograms: a sharp vertical stripe (fall impact) is an edge the network already knows how to detect; a repeating vertical texture (jogging rhythm) is a periodic pattern; a uniform low-energy region (sitting still) is a flat blob. Transfer learning here exploits the fact that the underlying visual primitives transfer even though the input is no longer a natural photograph.

Two-stage fine-tuning. A new classification head (Dropout + Dense(128) + Dropout + output layer with the appropriate number of classes) is appended to the pretrained backbone. In stage 1, the backbone is frozen and only the head is trained at learning rate 10^{-3} for up to 12 epochs. In stage 2, the top layers of the backbone are unfrozen — the top 30 layers for ResNet50 and the top 8 layers for VGG16 — and the network is fine-tuned end-to-end at the much smaller learning rate 10^{-5} for up to 25 epochs. The reduced learning rate prevents catastrophic forgetting of the pretrained features while still allowing the upper layers to specialize to the spectrogram domain.

Table 15: Transfer learning results.

Model	Task	Accuracy	F1-Score
ResNet50	Binary	0.9791	0.9778
ResNet50	Fall4	0.6505	0.6511
ResNet50	Multi13	0.7804	0.7548
VGG16	Binary	0.9895	0.9889
VGG16	Fall4	0.7059	0.7049
VGG16	Multi13	0.8261	0.8152

Path D Confusion Matrices: ResNet50 (top) vs VGG16 (bottom)

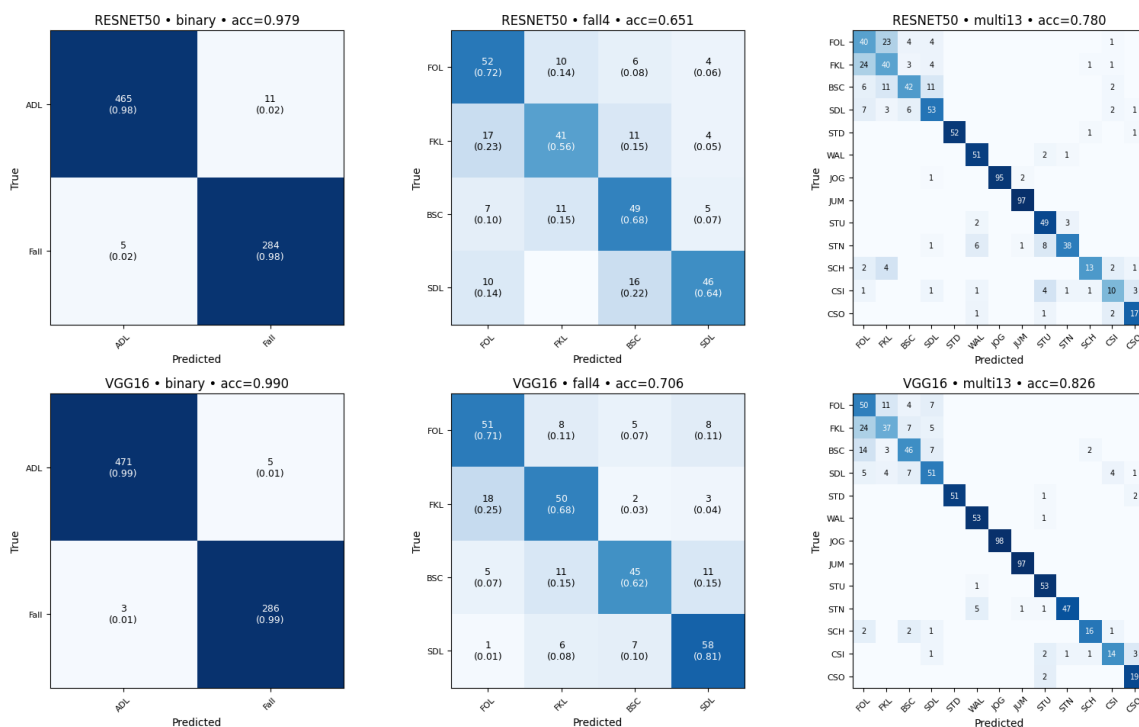


Figure 9: Confusion matrices for spectrogram-based transfer learning. Top row: ResNet50 on binary, fall4, and multi13. Bottom row: VGG16 on the same three tasks.

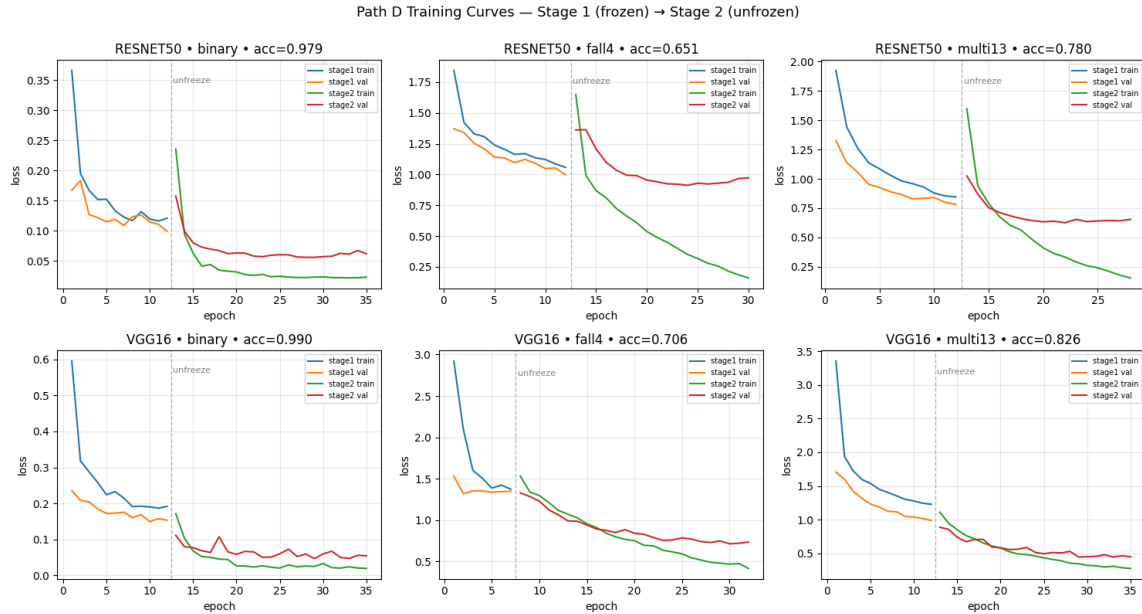


Figure 10: Training and validation loss curves for spectrogram-based transfer learning. The dashed vertical line marks the transition from stage 1 (frozen backbone, head only) to stage 2 (top layers unfrozen, end-to-end fine-tuning). Stage 2 produces a clear additional drop in validation loss for the binary task, while ResNet50 shows mild overfitting on fall4 due to its higher capacity on a smaller dataset.

Findings. Three observations emerge. First, VGG16 outperforms ResNet50 on every task — by 1.0% on binary, 5.5% on fall4, and 4.6% on multi13. This is consistent with the nature of the input: spectrograms are low-resolution and visually simple compared to natural photographs, and ResNet50’s far greater depth offers no advantage when the texture is this constrained. The training curves in Figure 10 confirm this — ResNet50 begins overfitting on fall4 once unfrozen, while VGG16 continues to improve. Second, both spectrogram models outperform UCI HAR pretraining on the harder fall-type and 13-class tasks, suggesting that even a domain-mismatched pretraining (photographs of natural objects) provides more transferable features than a sensor-only pretraining that actively suppresses fall-like signatures. Third, neither spectrogram model surpasses the best raw-signal CNN on binary fall detection. Together, these results indicate that for this dataset, direct learning from raw multichannel signals remains the most effective approach, while spectrogram-based transfer learning provides a competitive alternative on harder tasks where labeled data is limited.

Figure 11 summarizes the three transfer-learning strategies against the from-scratch CNN baseline across all three tasks. The from-scratch CNN remains the strongest model on every task, but the gap narrows on the harder problems where transfer learning has more room to contribute. VGG16 emerges as the strongest transfer-based model on binary classification (98.95%) and the fall-type task (70.6%), while UCI HAR transfer is only competitive on binary detection.

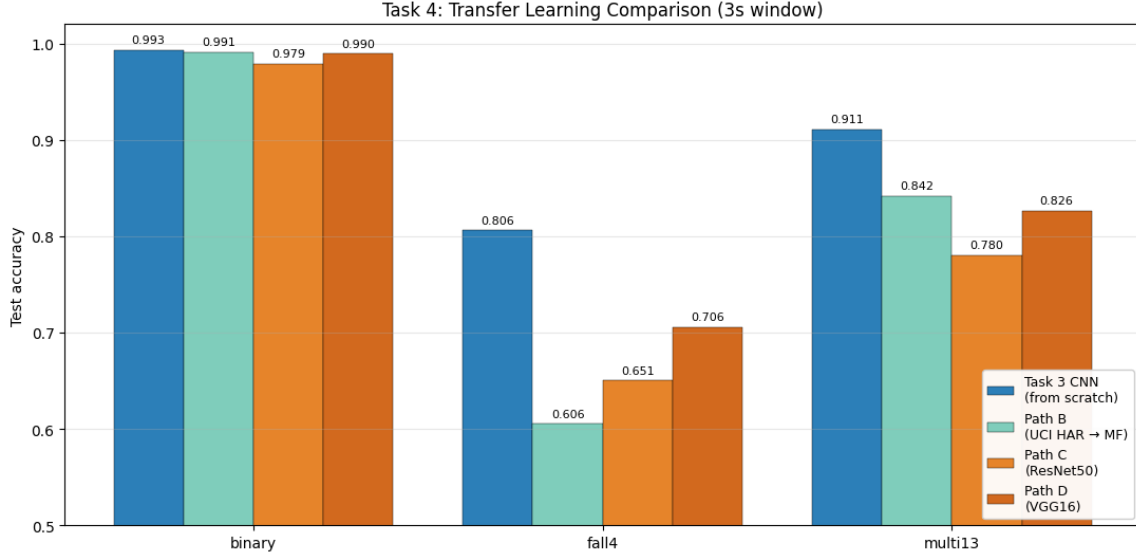


Figure 11: Test accuracy for four modeling approaches

6 Evaluation and Comparative Analysis

6.1 Validation Protocols

To evaluate robustness beyond a standard train-test split, we used two stricter validation strategies.

Leave-One-Subject-Out (LOSO): all windows belonging to one subject are held out for testing, while the model is trained on all remaining subjects. This is repeated for each subject. LOSO therefore tests whether the model generalizes to unseen individuals.

Leave-One-Event-Out (LOEO): one complete fall event is held out for testing at a time. In this work, an event is defined as a unique tuple of subject, activity, and trial number. The model is trained on all remaining events and evaluated on the held-out event. LOEO therefore tests whether the model generalizes to unseen fall occurrences.

Based on the dataset organization used in this project, LOSO produced 24 folds, corresponding to the 24 available subjects. LOEO produced 288 folds, since each of the 24 subjects contributes 4 fall activities with 3 trials each, resulting in $24 \times 4 \times 3 = 288$ fall-event folds.

6.2 Comparison Across Standard Split, LOSO, and LOEO

Table 16: CNN performance across evaluation protocols for binary fall detection

Protocol	Accuracy	F1-Score	Precision	Recall	Folds
Standard split	0.9935	0.9931	–	–	1
LOSO	0.9814	0.9684	0.9555	0.9868	24
LOEO	0.9917	0.9951	1.0000	0.9917	288

Table 16 shows that the CNN remains highly accurate under all evaluation settings. As expected, LOSO is the most challenging setting because the test subject is completely unseen during training. LOEO produces slightly stronger results because only one event is held out at a time, while other data from the same subject may still appear in training. Together, these results indicate that the proposed CNN is robust both to unseen events and to subject-level variation.

6.3 Comparison of Machine Learning and Deep Learning

Table 17: Comparison of ML and DL models for binary fall detection.

Family	Model	Accuracy	F1-Score
Classical ML	XGBoost	0.9509	0.9562
Deep Learning	ANN (3 s)	0.9699	0.9677
Deep Learning	CNN (3 s)	0.9935	0.9931
Deep Learning	LSTM (3 s)	0.9516	0.9494
Transfer Learning	VGG16 Spectrogram	0.9895	0.9889

Table 17 shows that feature engineering provides a strong baseline, but it also limits the model to predefined signal summaries. In contrast, deep learning models operate directly on the raw multichannel signal and can learn task-specific representations from data. This is especially beneficial in fall detection, where discriminative patterns often appear as short-duration peaks, transitions, and cross-sensor interactions that may be weakened during handcrafted feature extraction. Therefore, the superior performance of CNN and transfer learning models suggests that representation learning is more effective than manual feature design for the MobiFall binary fall detection task.

6.4 Error Analysis

Bringing together the confusion matrices reported throughout this study, three patterns emerge that are common across model families and that highlight the genuine difficulty of fall detection beyond aggregate accuracy.

Binary task - best separation. On binary fall detection, all strong models (CNN, ResNet50/VGG16 spectrogram, and the UCI HAR transfer model) achieve over 97% accuracy with errors roughly balanced between false alarms and missed falls. The best from-scratch CNN at 3s makes only 5 misclassifications out of 765 test windows (Figure 2), confirming that fall vs ADL is a relatively easy task once enough temporal context is provided.

Fall-type task - the FOL/FKL boundary is the most difficult mode. Across every architecture, the FOL (forward-lying) and FKL (front-knees-lying) classes are heavily confused with one another (Figures 3 and 9). Both are forward falls from a standing position and differ only in which body part contacts the ground first, a sub-second event difficult to localize from inertial signals alone. Even the best CNN misclassifies roughly 22% of FKL samples as FOL. In contrast, BSC (back-sitting-chair) and SDL (sideward-lying) are recognized with much higher accuracy because their kinematic signatures are distinctive: BSC involves a backward deceleration while SDL has a dominant lateral acceleration component. This indicates that the limiting factor for fall-type classification on MobiFall is not model capacity but the inherent ambiguity of forward-fall kinematics in pocket-mounted sensing.

13-class task - Moderately difficult. ADL classes with continuous, distinctive motion (walking, jogging, jumping) achieve 95–100% recall (Figure 4). The remaining errors concentrate in three places: the FOL/FKL confusion described above; occasional confusion between stairs-up and stairs-down (STU/STN), which share a similar repetitive vertical pattern; and lower performance on the under-represented short-duration classes (SCH, CSI, CSO each contribute only ~ 22 test samples), where the model has fewer examples to learn from.

Implications for deployment. For a real-world fall-detection system, the binary task is the most important and the most reliable. Distinguishing fall types is desirable for downstream response (e.g., alerting the appropriate emergency action), but accurate fine-grained classification of forward-fall subtypes likely requires either richer sensing (e.g., body-mounted IMUs) or longer post-impact context than 3-second windows provide.

7 Member Contributions

Task	Mrunal	Sushant
Introduction & Methodology	Assisted in writing	Assisted in writing
Literature Review	Summarized 5 research papers and contributed to comparative review table	Summarized 5 research papers and contributed to comparative review table
Deep Learning Implementation	Data Processing & Feature preparation	Developed ANN/CNN/LSTM models
Advanced Modeling & Transfer Learning	Performed result analysis of transfer learning outcomes	Implemented transfer learning models and fine-tuning experiments
Evaluation & Comparative Analysis	Designed extraction and selection pipeline	Generated plots, confusion matrices, and carried out error analysis
Results & Conclusion	Assisted in writing	Assisted in writing

Table 18: Summary of Member Contributions

8 Results and Conclusion

This project explored smartphone-based fall detection and activity recognition using the MobiFall Dataset v2.0. Different modeling approaches were studied, including classical machine learning, deep learning, and transfer learning, along with stricter evaluation methods such as LOSO and LOEO. The overall study showed that learning directly from raw inertial sensor data is more effective than relying only on handcrafted features, especially for capturing the temporal patterns involved in fall events.

An important lesson from this work is that model performance depends not only on the choice of algorithm, but also on preprocessing, window design, and evaluation strategy. Deep learning methods provided clear improvements over classical machine learning by automatically learning useful patterns from the sensor signals. At the same time, the experiments also showed that strong performance on a standard split does not always guarantee equally strong generalization to unseen subjects or events, which is especially important in wearable AI applications.

From a practical perspective, this study suggests that smartphone and wearable sensor data can be used effectively for fall detection when combined with suitable deep learning models and careful validation. Such systems have strong potential in health monitoring and assisted living, where reliable and timely fall detection can improve safety and response. In future work, the system can be extended to more realistic datasets, lighter deployment-ready models, and more advanced architectures to further improve robustness in real-world settings.

References

- [1] G. Vavoulas, M. Pediaditis, C. Chatzaki, E. G. Spanakis, and M. Tsiknakis, “The MobiFall dataset: Fall detection and classification with a smartphone,” *International Journal of Monitoring and Surveillance Technologies Research*, vol. 2, no. 1, pp. 44–56, 2014.
- [2] H. Yhdego, C. Paolini, and M. Audette, “Toward real-time, robust wearable sensor fall detection using deep learning methods: A feasibility study,” *Applied Sciences*, vol. 13, no. 8, p. 4988, 2023.
- [3] J. Liu, X. Li, S. Huang, R. Chao, Z. Cao, S. Wang, A. Wang, and L. Liu, “A review of wearable sensors based fall-related recognition systems,” *Engineering Applications of Artificial Intelligence*, vol. 121, p. 105993, 2023.
- [4] J.-K. Kim, K. Lee, and S. G. Hong, “Detection of important features and comparison of datasets for fall detection based on wrist-wearable devices,” *Expert Systems with Applications*, vol. 234, p. 121034, 2023.
- [5] V. Fula and P. Moreno, “Wrist-based fall detection: Towards generalization across datasets,” *Sensors*, vol. 24, no. 5, p. 1679, 2024.
- [6] J. Zhang, Z. Li, Y. Liu, J. Li, H. Qiu, M. Li, G. Hou, and Z. Zhou, “An effective deep learning framework for fall detection: Model development and study design,” *Journal of Medical Internet Research*, vol. 26, p. e56750, 2024.
- [7] S. T. Haque, M. Debnath, A. Yasmin, T. Mahmud, and A. H. H. Ngu, “Experimental study of long short-term memory and transformer models for fall detection on smartwatches,” *Sensors*, vol. 24, no. 19, p. 6235, 2024.
- [8] A. Leone, A. Manni, G. Rescio, P. Siciliano, and A. Caroppo, “Deep transfer learning approach in smartwatch-based fall detection systems,” *Engineering Proceedings*, vol. 78, no. 1, p. 2, 2024.
- [9] A. Yasmin, T. Mahmud, S. T. Haque, S. Alamgeer, and A. H. H. Ngu, “Enhancing real-world fall detection using commodity devices: A systematic study,” *Sensors*, vol. 25, no. 17, p. 5249, 2025.
- [10] M. Villa and E. Casilari, “The impact of the accelerometer sampling rate on the performance of machine and deep learning models in wearable fall-detection systems,” *Sensors*, vol. 26, no. 1, p. 162, 2026.